

PHP Assessment Test

Technical Assessment: Airline Voucher Seat Assignment Application

Stack: React + PHP Laravel (Latest)

Duration: 3 - 5 Hours

Difficulty: Intermediate

Objective

Build a **Voucher Seat Assignment** web application for an airline promotional campaign.

For each flight, the airline needs to randomly assign **3 unique seat numbers** to voucher winners. The seat map varies depending on the aircraft type. A crew member will use this application to:

1. Input flight and crew details.
 2. Generate exactly 3 random, non-repeating seats based on the aircraft layout.
 3. Prevent duplicate voucher assignments for the same flight number on the same date.
 4. Persist the assignment records to a database.
-

Requirements

Frontend (React)

Build a user interface that allows the crew to enter the following information:

Field	Type	Notes
Crew Name	Text input	Required
Crew ID	Text input	Required
Flight Number	Text input	Required (e.g., GA102)
Flight Date	Date input	Format: DD-MM-YYYY

Field	Type	Notes
Aircraft Type	Dropdown	Options: <code>ATR</code> , <code>Airbus 320</code> , <code>Boeing 737 Max</code>

Behavior

- A **"Generate Vouchers"** button triggers the following flow:
 1. Call `POST /api/check` to validate whether the given flight already has assigned vouchers for that date.
 2. If vouchers **do not exist**:
 - Call `POST /api/generate` to create and store the assignment.
 - Display the 3 randomly chosen seat numbers on screen.
 3. If vouchers **already exist**:
 - Display a clear error message indicating that vouchers have already been generated for that flight and date.

Backend (PHP Laravel)

Implement a REST API using **Laravel** (latest stable version) with the following two endpoints:

`POST /api/check`

Checks whether voucher assignments already exist for a given flight and date.

Request Body:

```
{
  "flightNumber": "GA102",
  "date": "2025-07-12"
}
```

Response Body:

```
{
  "exists": true
}
```

`POST /api/generate`

Generates 3 random unique seats based on the aircraft layout, saves the record to the database, and returns the result.

Request Body:

```
{
  "name": "Sarah",
  "id": "98123",
  "flightNumber": "ID102",
  "date": "2025-07-12",
  "aircraft": "Airbus 320"
}
```

Response Body:

```
{
  "success": true,
  "seats": ["3B", "7C", "14D"]
}
```

Laravel-Specific Guidelines

- Define routes in `routes/api.php`.
- Use **Laravel Form Request** classes for input validation.
- Use **Eloquent ORM** for all database operations.
- Implement seat generation logic in a dedicated **Service class** (e.g., `App\Services\SeatGeneratorService`).
- Use **Laravel Resource** classes for consistent API response formatting.
- Handle errors using Laravel's built-in exception handler or custom exceptions.

Database (SQLite)

Use **SQLite** as the database driver. Configure the connection in Laravel's `.env` file:

```
DB_CONNECTION=sqlite
DB_DATABASE=/absolute/path/to/database/vouchers.db
```

Migration Schema — `vouchers` table:

```
Schema::create('vouchers', function (Blueprint $table) {
    $table->id();
    $table->string('crew_name');
    $table->string('crew_id');
    $table->string('flight_number');
    $table->string('flight_date');
    $table->string('aircraft_type');
    $table->string('seat1');
    $table->string('seat2');
    $table->string('seat3');
    $table->timestamps();
});
```

Field	Type	Constraint
<code>id</code>	BIGINT	PRIMARY KEY, AUTO INCREMENT
<code>crew_name</code>	VARCHAR	NOT NULL
<code>crew_id</code>	VARCHAR	NOT NULL
<code>flight_number</code>	VARCHAR	NOT NULL
<code>flight_date</code>	VARCHAR	NOT NULL
<code>aircraft_type</code>	VARCHAR	NOT NULL
<code>seat1</code>	VARCHAR	NOT NULL
<code>seat2</code>	VARCHAR	NOT NULL
<code>seat3</code>	VARCHAR	NOT NULL
<code>created_at</code>	TIMESTAMP	Auto-managed by Laravel
<code>updated_at</code>	TIMESTAMP	Auto-managed by Laravel

Note: Use Laravel's migration system (`php artisan make:migration`) to create the table. Do **not** write raw SQL for schema creation.

Seat Layout Reference

The following seat maps define the valid seat combinations for each aircraft type:

Aircraft Type	Row Range	Seats per Row	Example Seats
ATR	1–18	A, C, D, F	1A , 18F
Airbus 320	1–32	A, B, C, D, E, F	1A , 32F
Boeing 737 Max	1–32	A, B, C, D, E, F	1A , 32F

Note: The generated seats must be valid according to the aircraft type selected. For example, seat 5B is **not valid** for an ATR aircraft.

Evaluation Criteria

Criteria	Weight
Functional correctness (API + UI works end-to-end)	35%
Laravel best practices (routing, validation, Eloquent, services)	25%
Code structure and modularity	15%
Input validation and error handling	15%
Code readability and documentation	10%

Bonus Points

- Use **Laravel Form Request** for validation with custom error messages.
- Implement a **unique composite constraint** on `flight_number` + `flight_date` at the database level.
- Write **Feature Tests** using Laravel's built-in testing tools (`php artisan test`).
- Use **Laravel API Resources** for consistent JSON response formatting.
- Provide a `Dockerfile` or `docker-compose.yml` for easy setup (Laravel Sail is also acceptable).

Deliverables

```
project/
├── frontend/      # React application
└── backend/       # Laravel application
```

```

|   |   └─ app/
|   |       └─ Http/
|   |           └─ Controllers/
|   |               └─ Requests/
|   |           └─ Models/
|   |               └─ Services/
|   └─ database/
|       └─ migrations/
|   └─ routes/
|       └─ api.php
|   └─ tests/
└─ README.md                # Instructions to build and run
└─ docker-compose.yml       # (Optional) Docker setup

```

README must include:

1. Prerequisites (PHP 8.2+, Composer, Node.js, etc.)
2. Steps to install dependencies (`composer install` , `npm install`)
3. Steps to configure environment (`.env` setup, `php artisan key:generate`)
4. Steps to run migrations (`php artisan migrate`)
5. Steps to run the backend (`php artisan serve`) and frontend (`npm run dev`)
6. (Optional) Docker / Laravel Sail instructions

Important Notes

- Use the **latest stable version of Laravel** (11.x or above).
- You may use any React UI library (e.g., Tailwind CSS, Ant Design, Material UI).
- Laravel's built-in SQLite support is sufficient — no additional database server is required.
- Ensure CORS is properly configured for the React frontend to communicate with the Laravel API.
- Ensure the application runs without errors when following your provided instructions.

Good luck!